Technical Note

What this is

A chat agent that answers questions about a PDF you upload. Every answer cites the exact page and a verbatim quote from the source. If the answer isn't in the document, the agent refuses and points to the closest related passage instead of guessing.

The pipeline

When you upload a PDF, two things happen:

1. **PyMuPDF** extracts text page by page. I split text on paragraph breaks, then greedily pack paragraphs into chunks of about 500 tokens with 60 tokens of overlap. The chunker falls back to a token-window split for any paragraph that doesn't fit. Every chunk carries its page number.
2. The chunks get embedded with **OpenAI `text-embedding-3-large`** (truncated to 1024 dimensions via Matryoshka — same retrieval quality, smaller storage, faster search) and upserted into **Pinecone** serverless. Chunk IDs are deterministic SHA-1 of `(doc_id, page, index)` so re-uploading the same PDF overwrites instead of duplicating.

When you ask a question, a **LangGraph** state machine runs five nodes:

1. **rewrite** — `gpt-4o-mini` rewrites the query as a standalone search query using the last few turns of conversation. Skipped on the first turn.
2. **retrieve** — top-20 chunks from Pinecone, filtered by `doc_id`.
3. **rerank** — **Cohere `rerank-3.5`** rescores the 20 candidates and keeps the top 5. If Cohere rate-limits us (the trial key is 10 RPM), the node falls back to the original Pinecone order without crashing.
4. **generate** — `gpt-4o` produces a structured output (`AgentAnswer` Pydantic model) with the answer text, a list of citations (`{page, quote}`), and a refusal flag. The system prompt forces verbatim quotes and same-language replies.
5. **verify** — every quote is checked against the retrieved chunks. The check normalises whitespace and hyphenation, tries strict substring first, then falls back to bag-of-words token overlap (≥ 85% match). If any quote doesn't ground, the answer is replaced with a refusal that points to the closest retrieved passage.

The frontend is **Next.js 15** with **react-pdf**. Citations are clickable chips that jump the viewer to

the cited page and highlight the matching text in yellow. The answer streams token by token over Server-Sent Events — the backend forwards `on_chat_model_stream` chunks during generate, the frontend incrementally parses partial JSON to extract the growing `answer` field.

Why these choices

PyMuPDF over Docling. Docling gives layout-aware parsing with bounding boxes, but it took ~70 seconds per ingest on my machine and is heavy to deploy. PyMuPDF parses the same 5-page PDF in 50 ms with a wheel that's a few MB. The trade-off: I lose tables and figures as structured objects. For text-heavy PDFs that's fine; for tabular reports I'd add a fallback path.

OpenAI embeddings over BGE-M3. I started with multilingual `BGE-M3` via fastembed, but the install pulled in PyTorch and a 2 GB model download, and BGE-M3 isn't natively supported by fastembed (had to fall back to `intfloat/multilingual-e5-large`). OpenAI is one HTTP call, no model download, and the multilingual quality is good enough that Hindi/Marathi queries on an English doc still work end-to-end. Trade-off: I'm dependent on the OpenAI API.

Pinecone over Qdrant. Qdrant is excellent and self-hosted, but I had to maintain a Docker container, a volume, and a port mapping just for one small index. Pinecone serverless is a managed URL — one less thing to break in production. Trade-off: vendor lock-in. The schema is portable so a swap is possible.

Cohere reranker. Anthropic's research shows reranking cuts retrieval failure rate by roughly 3× when stacked on top of dense retrieval. With a small PDF this matters less (we already retrieve all 5 chunks), but for any real document it's the single biggest quality lever after the embedding model.

Single-vector retrieval + reranker, not ColBERT or hybrid BM25. ColBERT roughly 100× the storage cost. BM25 + dense fusion would help on rare-token queries but adds index complexity. With reranker on top, single-vector dense was enough for our eval.

`gpt-4o` for generate, `gpt-4o-mini` for rewrite. The generate step is doing the actual reasoning + structured output and benefits from the bigger model. The rewrite step just resolves coreferences in 1-2 sentences and works fine on the smaller, ~10× cheaper model.

Substring grounding instead of an NLI model or LLM judge at runtime. I evaluated MiniCheck and faithfulness-as-a-judge and decided against both for the runtime path: NLI adds a 150-200 ms hop, and an LLM judge adds a full extra LLM call. The substring + bag-of-words check runs in microseconds and catches the failure mode I actually care about — fabricated quotes. For *answer-level* faithfulness I run Ragas in the eval suite, which uses an LLM judge but offline.

Strict refusal contract. The system prompt says "if you can't ground a claim verbatim, refuse." The refusal template names the closest retrieved passage with a page number so the user has somewhere to go. This is the NotebookLM pattern and beats a flat "I don't know."

LangSmith over self-hosting. Setting `LANGSMITH_TRACING=true` plus a key turns on per-node tracing automatically because LangGraph reports to LangSmith natively. Each turn shows latency, token counts, cost, inputs, and outputs at every node. I wired this in `app/__init__.py` so any module that imports from `app.*` gets it.

Eval results

Two suites. The first is a binary pass/fail check on a curated set; the second is Ragas with judge-LLM scoring.

Binary suite (11 queries · `make eval`)

5 valid English + 3 out-of-scope + 3 multilingual (Hindi × 2, Marathi × 1) on a 5-page assignment PDF.

```
Valid:        8/8 pass (100%)
Invalid:      3/3 pass (100%)
Multilingual: 3/3 pass (100%)
Overall:      11/11 pass (100%)
```

The multilingual cases test cross-lingual grounding: a Hindi query against an English source should produce a Hindi answer with English-source quotes. All three pass. The script lookup and the verifier both treat Devanagari correctly because the substring check uses Unicode-aware normalisation.

Ragas (8 queries · `make ragas`)

Same set, valid queries only (refusals don't have meaningful Ragas semantics).

```
faithfulness:        0.948
answer_relevancy:    0.533
context_precision:   0.951
context_recall:      1.000
```

`faithfulness 0.948` says ~95% of claims in answers are entailed by the retrieved context

— the verifier catches the rest. `context_precision 0.951` says when retrieval ranks a chunk in the top-K it's almost always actually relevant. `context_recall 1.000` says the answer's required information was in the retrieved chunks every single time.

`answer_relevancy 0.533` looks low but the metric is noisy. Ragas computes it by asking the judge to generate three reverse-questions from the answer and measuring cosine similarity to the original. `gpt-4o-mini` only returns one generation per call (`LLM returned 1 generations instead of requested 3` warnings during the run), so the metric runs on n=1 instead of n=3 and the variance is higher than designed. The fix is either a different judge LLM or making three sequential calls; for this submission I logged the limitation rather than chase it.

The full per-row report is at `evals/ragas_report.json`.

What I'd do next

- **Anthropic Contextual Retrieval** — prepend a 1-2 sentence LLM-generated context to each chunk before embedding. Anthropic's reported numbers: –67% retrieval failure when stacked with rerank. I built this in v1 then took it out to keep the v2 pipeline simple; the data clearly says to put it back.
- **CRAG-style self-correction** — score retrieval as `correct / ambiguous / insufficient` after rerank, and on `insufficient` rewrite the query and re-retrieve once. This would reduce false refusals on ambiguous phrasing.
- **Bigger eval set, generated with `RagasTestsetGenerator`** — 50 questions instead of 11. The current set is good for regression but small.
- **Per-claim NLI verifier** — substring is too permissive on heavy paraphrases that token-overlap above 85%. A small NLI model (MiniCheck) would catch this without the cost of a judge LLM.
- **Multi-doc support** — the data model already carries `doc_id`; the UI just needs a "library" view. About a day of work.
- **Auth + rate limiting** — the API is wide open. For a real deploy I'd put it behind Clerk/Auth0 and Slowapi.